Informatique Théorique

CM 10: Programmation dynamique (1)

Florian Bridoux

Polytech Nice Sophia

2025-2026

Table des matières

1 Introduction

2 Exemple de chevauchement : Chemin dans une grille

3 Mémoïsation

4 Tabulation

Définition de la programmation dynamique

La programmation dynamique est une méthode de résolution de problèmes qui s'applique lorsqu'un problème peut être décomposé en sous-problèmes (comme dans l'approche « diviser pour régner »), et que l'une au moins des propriétés suivantes est vérifiée :

- **Chevauchement des sous-problèmes** : certains sous-problèmes apparaissent plusieurs fois dans le processus de résolution ;
- **Structure optimale** : la solution optimale du problème global peut être construite à partir des solutions optimales de ses sous-problèmes.

Définition de la programmation dynamique

La programmation dynamique consiste à ne calculer qu'une fois chaque sous problème en le mémorisant et en ignorant les solutions non optimales des sous-problèmes dans le cas des problèmes d'optimisation.
Il y a deux approches de programmations dynamiques différentes : la **mémoïsation** et la **tabulation**.

Table des matières

1 Introduction

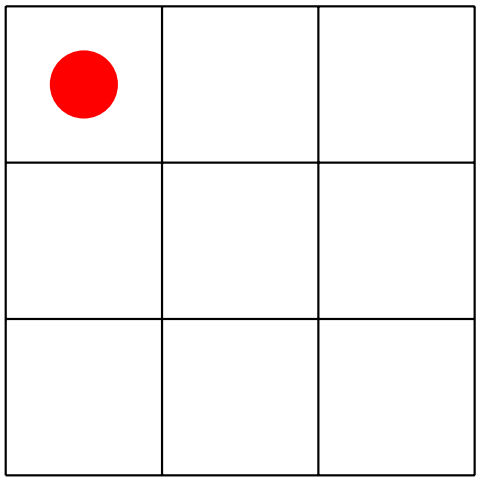
2 Exemple de chevauchement : Chemin dans une grille

3 Mémoïsation

4 Tabulation

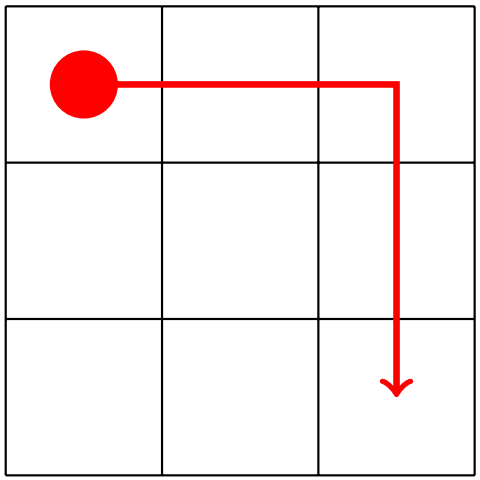
Exemple de chevauchement : Chemin dans une grille

Le problème est le suivant : on nous donne la hauteur y et la largeur x d'une grille et on nous demande le nombre de façons différentes d'aller de $(1, 1)$ à (y, x) en ne faisant que des mouvements vers la droite et vers le bas.



Exemple de chevauchement : Chemin dans une grille

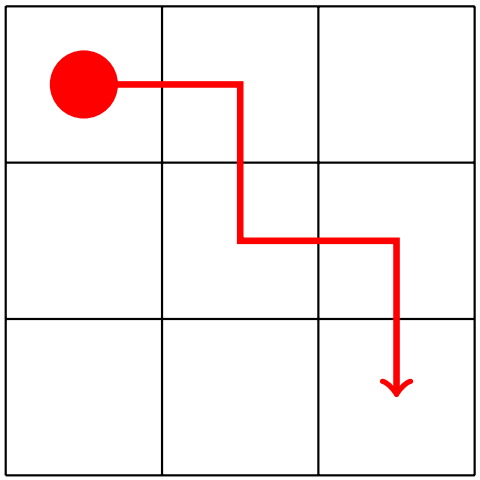
Le problème est le suivant : on nous donne la hauteur y et la largeur x d'une grille et on nous demande le nombre de façons différentes d'aller de $(1, 1)$ à (y, x) en ne faisant que des mouvements vers la droite et vers le bas.



- *ddbb*

Exemple de chevauchement : Chemin dans une grille

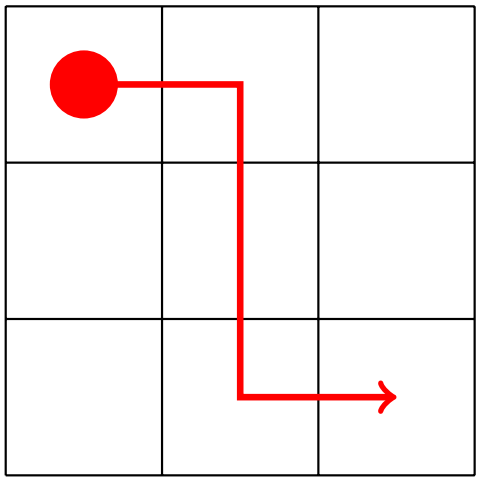
Le problème est le suivant : on nous donne la hauteur y et la largeur x d'une grille et on nous demande le nombre de façons différentes d'aller de $(1, 1)$ à (y, x) en ne faisant que des mouvements vers la droite et vers le bas.



- *ddbb*
- *dbdb*

Exemple de chevauchement : Chemin dans une grille

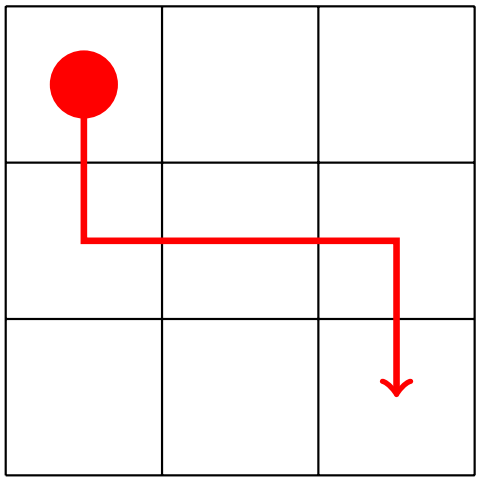
Le problème est le suivant : on nous donne la hauteur y et la largeur x d'une grille et on nous demande le nombre de façons différentes d'aller de $(1, 1)$ à (y, x) en ne faisant que des mouvements vers la droite et vers le bas.



- *ddbb*
- *dbdb*
- *dbbd*

Exemple de chevauchement : Chemin dans une grille

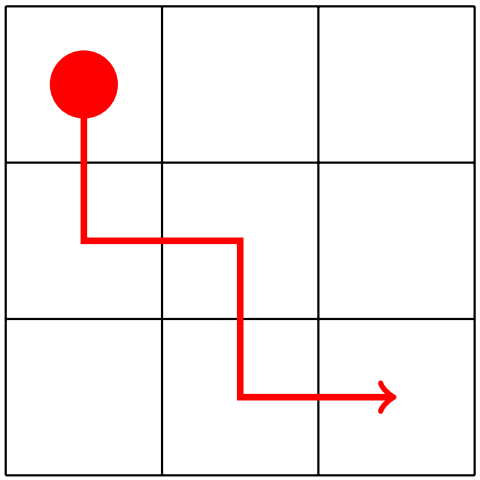
Le problème est le suivant : on nous donne la hauteur y et la largeur x d'une grille et on nous demande le nombre de façons différentes d'aller de $(1, 1)$ à (y, x) en ne faisant que des mouvements vers la droite et vers le bas.



- *ddbb*
- *dbdb*
- *dbbd*
- *bddb*

Exemple de chevauchement : Chemin dans une grille

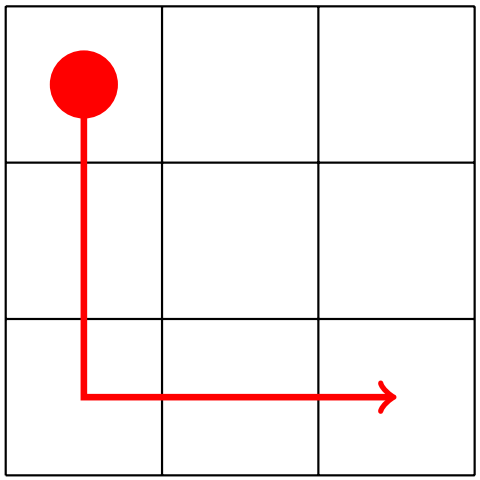
Le problème est le suivant : on nous donne la hauteur y et la largeur x d'une grille et on nous demande le nombre de façons différentes d'aller de $(1, 1)$ à (y, x) en ne faisant que des mouvements vers la droite et vers le bas.



- *ddbb*
- *dbdb*
- *dbbd*
- *bddb*
- *bdbd*

Exemple de chevauchement : Chemin dans une grille

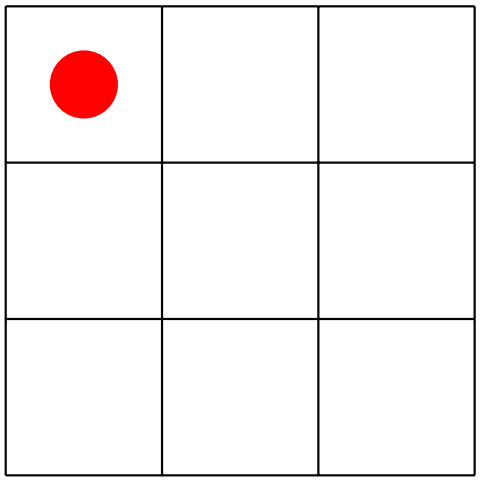
Le problème est le suivant : on nous donne la hauteur y et la largeur x d'une grille et on nous demande le nombre de façons différentes d'aller de $(1, 1)$ à (y, x) en ne faisant que des mouvements vers la droite et vers le bas.



- *ddbb*
- *dbdb*
- *dbbd*
- *bddb*
- *bdbd*
- *bbdd*

Exemple de chevauchement : Chemin dans une grille

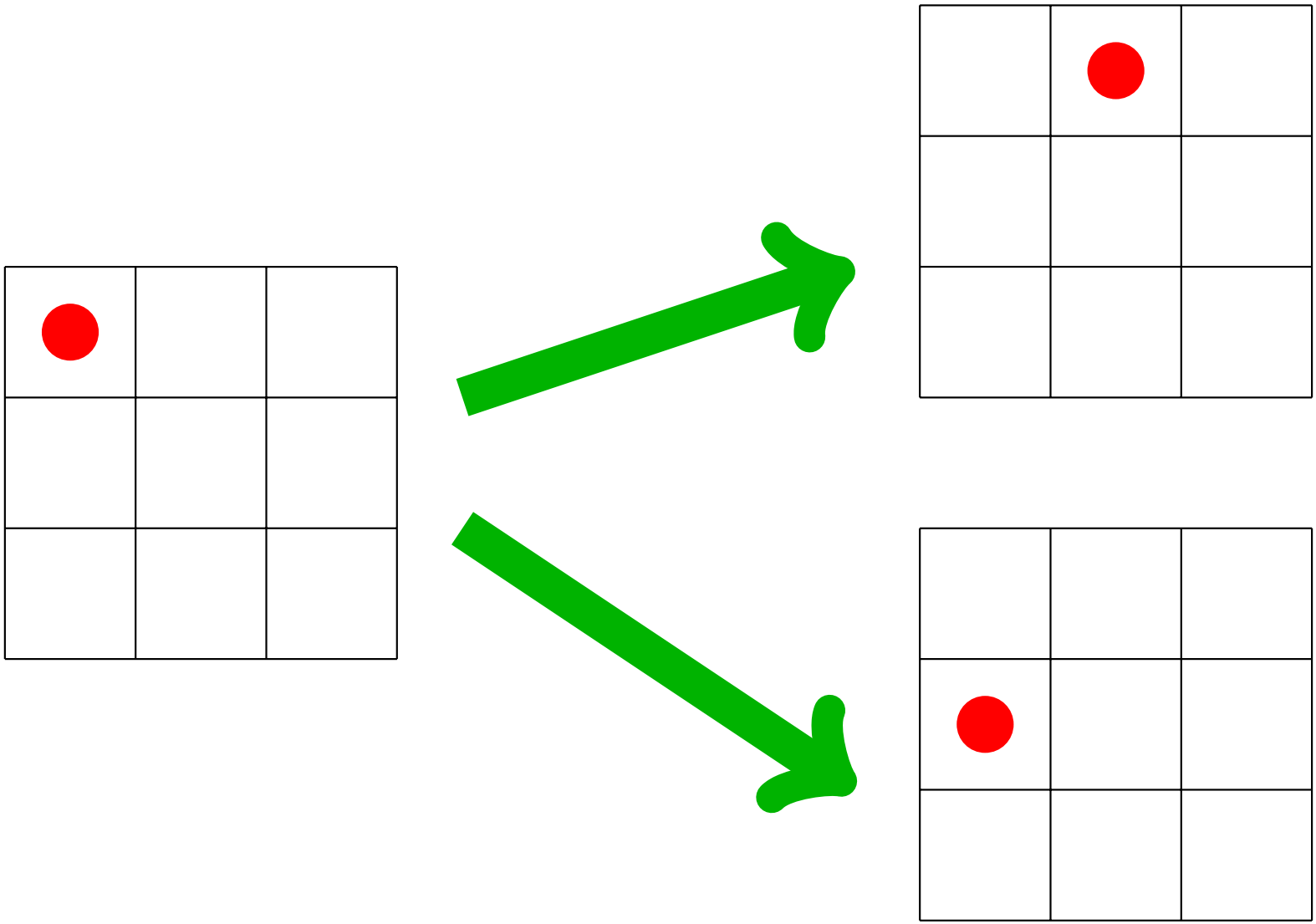
Le problème est le suivant : on nous donne la hauteur y et la largeur x d'une grille et on nous demande le nombre de façons différentes d'aller de $(1, 1)$ à (y, x) en ne faisant que des mouvements vers la droite et vers le bas.



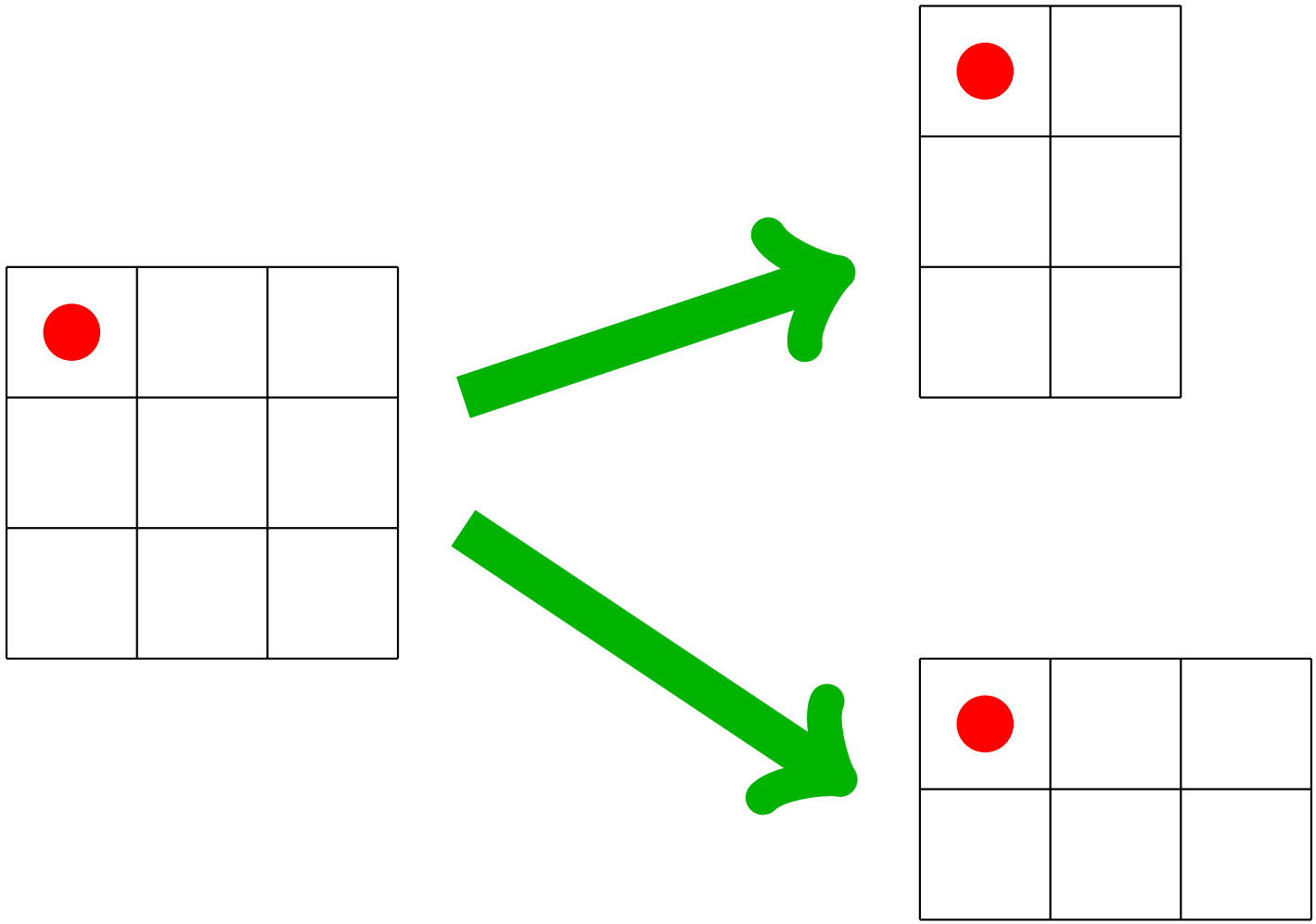
- *ddbb*
- *dbdb*
- *dbbd*
- *bddb*
- *bdbd*
- *bbdd*

Pour une grille de taille 3x3, il y a 6 possibilités !
Notons qu'énumérer les chemins pour les compter est très peu efficace : le nombre de chemin étant exponentiel en fonction de x, y...

Décomposition récursive du problème

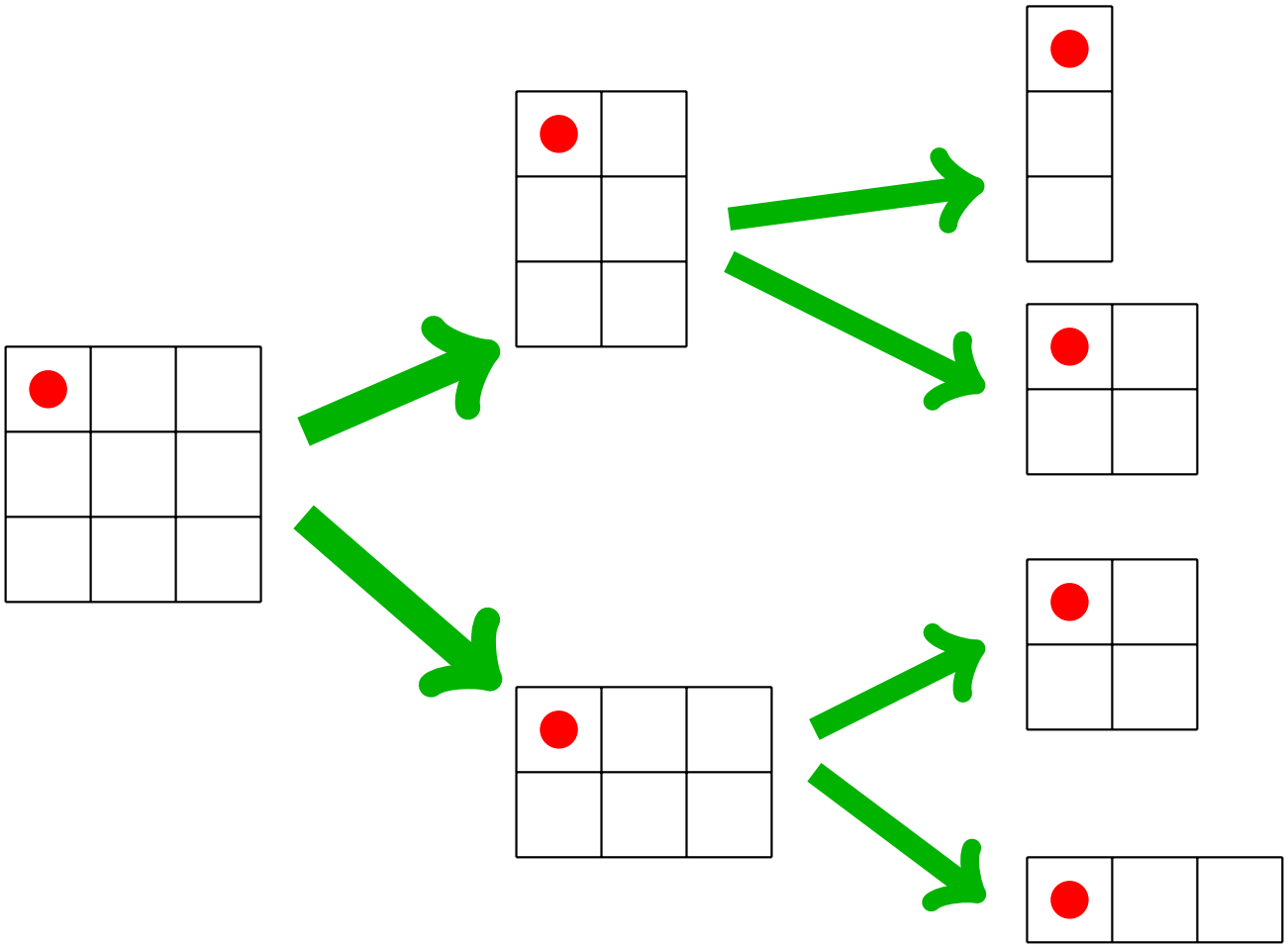


Décomposition récursive du problème



$$f(y,x) = f(y-1,x) + f(y,x-1).$$

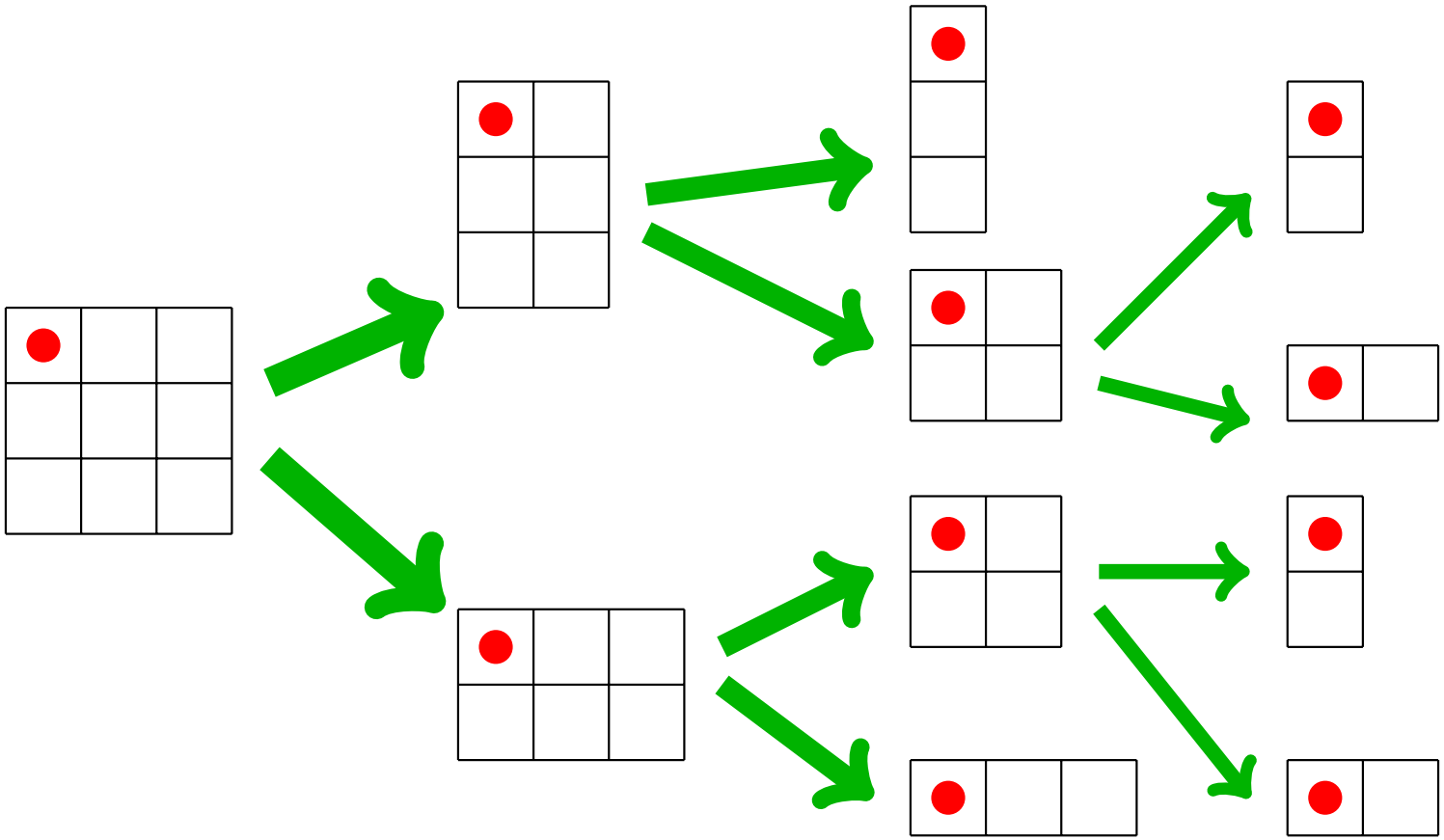
Décomposition récursive du problème



$$f(1,x) = f(y,1) = 1$$

$$\text{sinon}, f(y,x) = f(y-1,x) + f(y,x-1).$$

Décomposition récursive du problème



On remarque que le même sous problème (le nombre de chemin dans la grille 2x2) apparaît plusieurs fois !

Algorithme naïf pour compter les chemins

```
def f(y, x):
    # Sur le bord, un seul chemin
    if y == 1 or x == 1:
        return 1
    # Sinon bas + droite
    return f(y-1, x) + f(y, x-1)

print(f(300,300))
```

La complexité en temps de cette algorithme est (en gros) le nombre de chemins possibles (on peut montrer que c'est $\binom{x+y-2}{x-1} \approx \frac{2^{x+y-2}}{\sqrt{x+y-2}}$) soit exponentiel.
Sur l'exemple f(300,300), un programme naïf n'aura pas finit de calculer la fin de l'univers par exemple...

Table des matières

1. Introduction
2. Exemple de chevauchement : Chemin dans une grille
3. Mémoïsation
4. Tabulation

Algorithme naïf pour compter les chemins

La mémoïsation est simplement une approche où chaque fois qu'on résout un sous-problème, on mémorise la solution (dans une variable globale par exemple), pour ne pas le recalculer par la suite.

```
dct = {}

def f(y, x):
        if (y,x) not in dct:
            if y == 1 or x == 1:
                dct[(y,x)] = 1
        else:
                dct[(y,x)] = f(y-1, x) + f(y, x-1)
    return dct[(y,x)]

print(f(300,300))
```

Avec cette simple technique f(300,300) est calculé quasi instantanément.

Mémoïsation en Python : argument par défaut

En Python, on peut directement passer le dictionnaire comme valeur par défaut :

```
def f(y, x, dct={}):
    if (y, x) not in dct:
        if y == 1 or x == 1:
            dct[(y, x)] = 1
        else:
            dct[(y, x)] = f(y-1, x) + f(y, x-1)
    return dct[(y, x)]
f(300,300)
```

Comment ça marche ?

- La valeur par défaut `dct={}` est créée **une seule fois** lors de la définition de la fonction.
- Tous les appels récursifs partagent donc le même dictionnaire.
- **Attention :** avec des objets mutables en argument par défaut, le contenu persiste d'un appel à l'autre. Ici, c'est voulu !

Limites de la mémoïsation (top-down)

La mémoïsation garde la récursion **descendante** de la récursion naïve :

- on part d'un problème plus grand,
- puis on appelle récursivement des sous-problèmes plus petits.

Inconvénients :

1. Les appels récursifs ne sont pas gratuits. (Sauf optimisation spéciales comme les **récursions terminales** dans certains langages, mais pas pour ce problème et pas en Python.)
2. En Python, le nombre maximal de récursions est limité (par défaut `sys.getrecursionlimit()` ≈ 1000). **Un grand problème passe vite cette limite.**

Nous allons voir la technique de la **tabulation**, qui élimine la récursion et construit directement la table des résultats.

Table des matières

1. Introduction
2. Exemple de chevauchement : Chemin dans une grille
3. Mémoïsation
4. Tabulation

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5					
4					
3					
2					
1					
y/x	1	2	3	4	5

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1				
3	1				
2	1				
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1				
3	1				
2	1	2			
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1				
3	1				
2	1	2	3		
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1				
3	1				
2	1	2	3	4	
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1				
3	1				
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1				
3	1	3			
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1				
3	1	3	6		
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1				
3	1	3	6	10	
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1				
3	1	3	6	10	15
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1	4			
3	1	3	6	10	15
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1	4	10		
3	1	3	6	10	15
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1				
4	1	4	10	20	35
3	1	3	6	10	15
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1	5			
4	1	4	10	20	35
3	1	3	6	10	15
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1	5	15		
4	1	4	10	20	35
3	1	3	6	10	15
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1	5	15	35	
4	1	4	10	20	35
3	1	3	6	10	15
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Tabulation : initialisation de la grille

Comme c'est un problème à double entrée, on peut le représenter comme une matrice (une liste de liste, ou un tableau à deux dimensions). Ici les lignes représentent les valeurs de y, les colonnes, les valeurs de x et les cases, les valeurs de $f(y, x)$.

5	1	5	15	35	70
4	1	4	10	20	35
3	1	3	6	10	15
2	1	2	3	4	5
1	1	1	1	1	1
y/x	1	2	3	4	5

- $f(y, 1) = f(1, x) = 1$
- sinon, $f(y, x) = f(y, x - 1) + f(y - 1, x)$. (case de gauche + case du bas).

Algorithme par tabulation

La tabulation, consiste ici juste à remplir la grille et de récupérer la valeur qui nous intéresse.
On peut construire directement la grille en remplissant les cases de gauche à droite et de bas en haut.

```
def f(y, x):
    tab = [[0]*x for _ in range(y)]
    for i in range(1, y):
        tab[i][1] = 1
    for j in range(1, x):
        tab[1][j] = 1
    for i in range(2, y):
        for j in range(2, x):
            tab[i][j] = tab[i-1][j] + tab[i][j-1]
    return tab[y-1][x-1]
```

Algorithme par tabulation

Remarque : c'est une approche ascendante : on part des petits problèmes et à partir d'eux, on résout des problèmes de plus en plus gros.

Avantage :

- Complexité en temps : $\mathcal{O}(x \cdot y)$.
- Complexité en mémoire : $\mathcal{O}(x \cdot y)$, mais on pourrait optimiser à $\mathcal{O}(\min(x, y))$.
- Pas de récursion $\Rightarrow$ pas de limite de profondeur, pas de surcoût d'appels.

Désavantage :

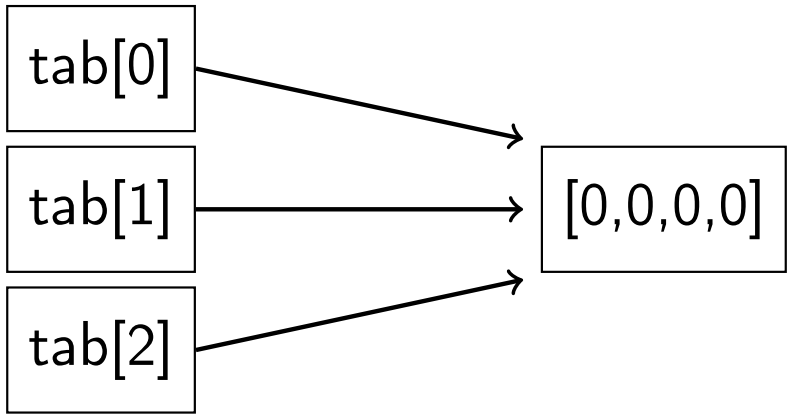
- Plus difficile à concevoir.
- Dans certain problèmes, il est difficile de savoir quels sont les cas de bases qu'on a vraiment besoin de calculer.

Digression : initialisation des matrices en Python

On pourrait penser à initialiser une matrice $y \times x$ en écrivant :

```
tab = [[0]*x]*y
```

Problème : toutes les lignes pointent vers la même liste en mémoire.

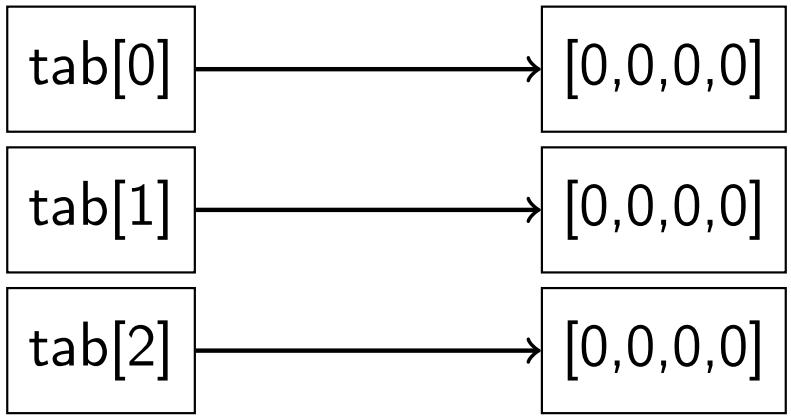


Conséquence : modifier une case d'une ligne modifie **toutes** les lignes !

Digression : initialisation des matrices en Python

La bonne méthode est de créer chaque ligne indépendamment :

```
tab = [[0]*x for _ in range(y)]
```



Chaque ligne est une liste différente $\rightarrow$ modifier une ligne ne modifie pas les autres.